

CS 422 – CS303 Language Theory & Finite Automata

Project Assignment 2025-2026

Real-Life Application of Automata Theory

Project Title: *Smart Home Security System Using Face Recognition*

	Student Name	Student ID
Group Members:	1- Jana shammary 2- Marwa abdulaziz 3- Alya Shehab 4- Maha Aljoaib 5- Maisa alzahrani	2240006642 2240000811 2240002062 2240006849 2240004660
Submission Date:	17 May	
Instructor:	Leena Awad alhusain	

3.1 Lexical Analyzer

The lexical analyzer identifies the valid tokens used in the language.
In this project, the tokens are:

D = detect face
V = verify face
U = unlock door
E = enter home

Each token is defined using ANTLR lexer rules to recognize valid input symbols.

D : 'd' ;
V : 'v' ;
U : 'u' ;
E : 'e' ;

The lexer checks the input sequence and recognizes valid symbols before passing them to the syntax analyzer.

Any undefined symbol is treated as a lexical error:

Input: dvxe

Error: x is an invalid token

3.2 Syntax Analyzer

The syntax analyzer checks whether the input sequence follows the correct order defined by the grammar rules.

The valid sequence in this project is:

$D \rightarrow V \rightarrow U \rightarrow E$

The grammar rule used in this project is:

start : D V U E EOF ;

The parser validates that the sequence strictly follows the grammar order before granting access to the smart home system.

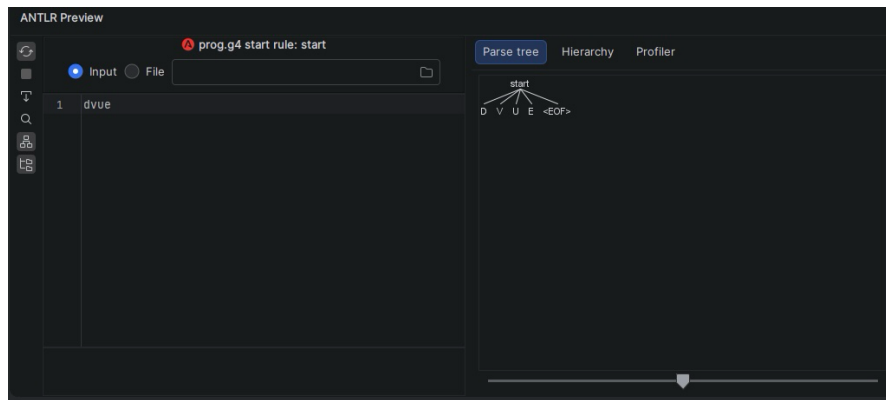
If the input follows the correct order, the parser accepts the sequence and generates a parse tree. Otherwise, the parser rejects the input and displays syntax errors.

3.3 Sample Runs

Test Case 1: Valid Input

The parser accepted the sequence because all tokens followed the correct grammar order.

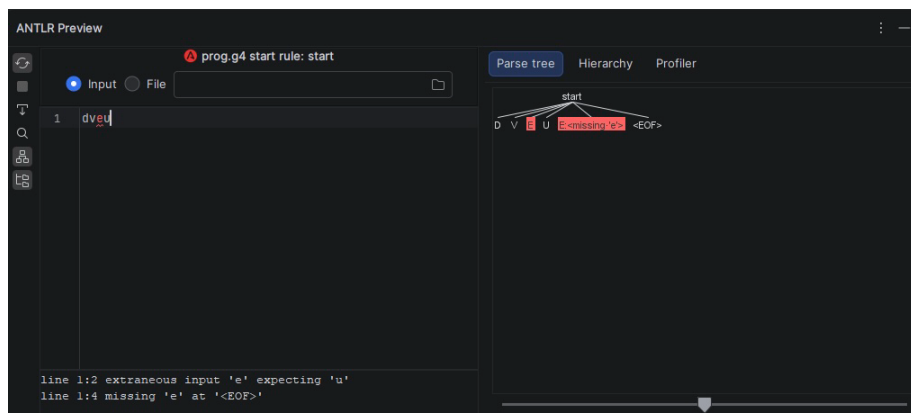
Component	Details
Input	dvue
Tokens	D V U E
Result	ACCEPTED
Error Message	None



Test Case 2: Invalid Input

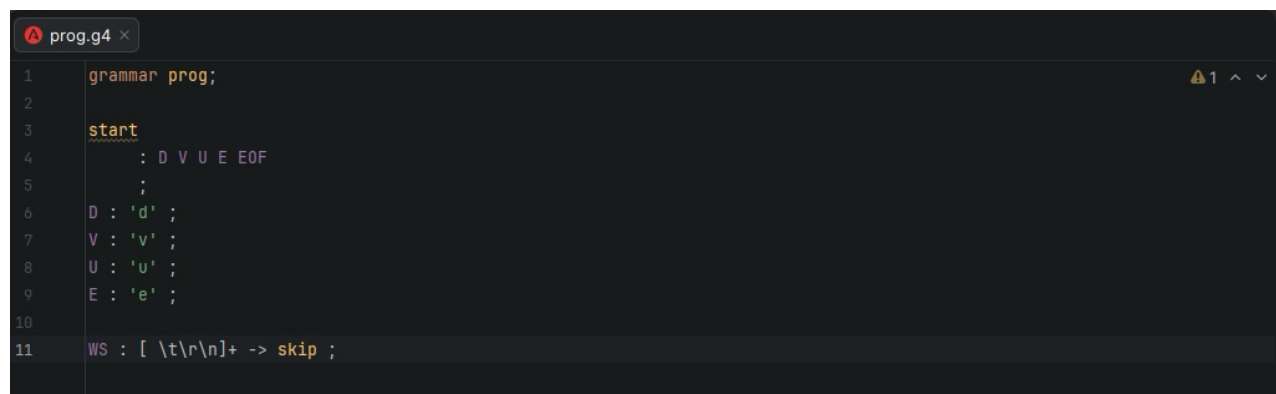
The parser rejected the sequence because token E appeared before token U.

Component	Details
Input	dveu
Tokens	D V E U
Result	REJECTED
Error Message	extraneous input 'e' expecting 'u'



Appendix: Source Code

```
grammar prog;  
  
start : D V U E EOF ;  
  
D : 'd' ;  
V : 'v' ;  
U : 'u' ;  
E : 'e' ;  
  
WS : [ \t\r\n]+ -> skip ;
```

A screenshot of a code editor window titled 'prog.g4'. The editor displays the following ANTLR grammar code:

```
1 grammar prog;  
2  
3 start  
4 : D V U E EOF  
5 ;  
6 D : 'd' ;  
7 V : 'v' ;  
8 U : 'u' ;  
9 E : 'e' ;  
10  
11 WS : [ \t\r\n]+ -> skip ;
```

The code is syntax-highlighted, with keywords like 'grammar', 'start', and 'EOF' in orange, and literals like 'd', 'v', 'u', 'e' in green. Line numbers 1 through 11 are visible on the left margin.

EOF ensures that no extra symbols appear after the valid sequence.

References

1. ANTLR Official Documentation – <https://www.antlr.org>
2. Course Slides and Lecture Notes
3. IntelliJ IDEA Documentation